Wholesale Medicine Database System

Group Members: Syed Soban Ahmed Shah , Muhammad Saad

Program: BS (Ai) “B”

Course: Database Lab

Instructor: Ali Hassan

Milestone	Date	Version	Remarks
ERD Diagram	4/26/2026	V1.0	
Schema Design	4/26/2026	V1.1	
Updated ERD	5/17/2026	V2.0	
Normalization	5/17/2026	V2.1	
Dataset Preprocessing	5/17/2026	V3.0	
Dataflow	5/17/2026	V3.1	
Database Setup (DDL)	5/17/2026	V4.0	
Data Population (DML)	5/17/2026	V5.0	

1. Entity Relationship Diagram (ERD)

Introduction:
The following ERD represents the database design of the Wholesale Medicine Distribution System. It shows the main entities, their attributes, and the relationships between them.

Entities Description:

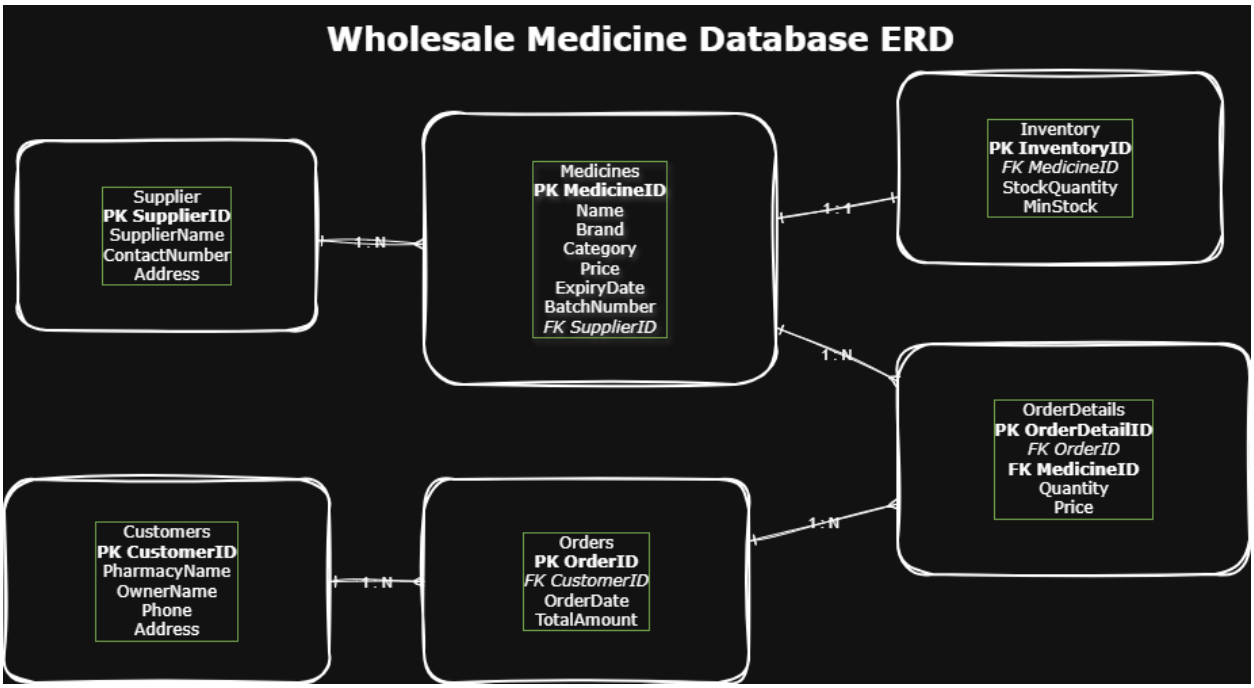
- **Supplier:** Stores information about medicine suppliers.

- **Medicines:** Contains details of all medicines including price and expiry.
- **Customers:** Stores pharmacy/customer details.
- **Orders:** Records customer orders.
- **OrderDetails:** Stores medicines included in each order.
- **Inventory:** Tracks stock levels of medicines.

Relationships:

- One supplier supplies many medicines (1:M)
- One customer can place many orders (1:M)
- One order contains multiple medicines (M:N via OrderDetails)
- One medicine can appear in many orders
- Each medicine has one inventory record (1:1)

Figure :



2. Relational Database Schema

The following relational schema is designed for the Wholesale Medicine Distribution System. Each table represents an entity, along with its attributes, primary keys (PK), and foreign keys (FK).

1. **Supplier**

```
Supplier(
    SupplierID PK,
    SupplierName,
    ContactNumber,
    Address
)
```

Description:

This table stores information about pharmaceutical companies or suppliers who provide medicines.

2. **Medicines**

```
Medicines(
    MedicineID PK,
    MedicineName,
    Brand,
    Category,
    Price,
    ExpiryDate,
    BatchNumber,
    SupplierID FK
)
```

Description:

This table contains details of all medicines available in the system. Each medicine is linked to a supplier.

3. **Customers**

```
Customers(
    CustomerID PK,
    PharmacyName,
    OwnerName,
    Phone,
    Address
)
```

Description:

This table stores information about customers, which are pharmacies or medical stores purchasing medicines.

4. **Orders**

```
Orders(
    OrderID PK,
    CustomerID FK,
    OrderDate,
    TotalAmount
)
```

Description:
This table records each order placed by customers. Each order is linked to a specific customer.

5. **Order Details**

```
OrderDetails(
    OrderDetailID PK,
    OrderID FK,
    MedicineID FK,
    Quantity,
    Price
)
```

Description:
This table stores the details of medicines included in each order. It resolves the many-to-many relationship between Orders and Medicines.

6. **Inventory**

```
Inventory(
    InventoryID PK,
    MedicineID FK,
    StockQuantity,
    MinimumStockLevel
)
```

Description:
This table tracks the stock levels of each medicine and helps identify low inventory.

Mileston 2

1.The ERD represents the complete relational structure of the Wholesale Medicine Database System including entities, relationships, primary keys, and foreign keys.

2. **NORMALIZATION SECTION**

Wholesale Medicine Database System — Normalization

First Normal Form (1NF)

Rule of 1NF

A table must:

Have atomic values

Have no repeating groups

Have unique rows

Supplier Table

Issue:

No repeating groups or multivalued attributes were found.

Justification:

Each supplier record contains atomic values such as SupplierName, ContactNumber, and Address. Every row is uniquely identified using SupplierID.

Result:

The table already satisfies 1NF. No changes were required.

Medicines Table

Issue:

The table already stores atomic values. Each medicine attribute contains only one value.

Justification:

Attributes such as MedicineName, Brand, Category, and Price contain single values only. No repeating groups exist.

Result:

The table satisfies 1NF without modification.

Customers Table

Issue:

No multivalued or repeating attributes were present.

Justification:

Each customer has unique and atomic details such as PharmacyName, OwnerName, and Phone.

Result:

The table already satisfies 1NF.

Orders Table

Issue:

The table contains atomic attributes only.

Justification:

OrderDate and TotalAmount store single values for each order.

Result:

The table satisfies 1NF.

OrderDetails Table

Issue:

No repeating groups exist.

Justification:

Each row stores one medicine entry per order using Quantity and Price attributes.

Result:

The table already satisfies 1NF.

Inventory Table

Issue:

No repeating groups or multivalued attributes were found.

Justification:

StockQuantity and MinimumStockLevel contain atomic values.

Result:

The table satisfies 1NF.

Second Normal Form (2NF)

Rule of 2NF

A table must:

Already be in 1NF

Have no partial dependency

Supplier Table

Issue:

No partial dependency exists because the table uses a single-column primary key.

Justification:

All attributes depend completely on SupplierID.

Result:

The table satisfies 2NF.

Medicines Table

Issue:

No partial dependency exists.

Justification:

All medicine attributes depend entirely on MedicineID.

Result:

The table already satisfies 2NF.

Customers Table

Issue:

No partial dependency exists.

Justification:

All customer attributes depend fully on CustomerID.

Result:

The table satisfies 2NF.

Orders Table

Issue:

No partial dependency exists.

Justification:

All order attributes depend on OrderID.

Result:

The table satisfies 2NF.

OrderDetails Table

Issue:

Potential duplication between Orders and Medicines needed to be resolved.

Change Made:

A separate OrderDetails table was created to manage the many-to-many relationship between Orders and Medicines.

Justification:

This prevents partial dependency and stores order-specific medicine data efficiently.

Result:

The table satisfies 2NF after normalization.

Inventory Table

Issue:

No partial dependency exists.

Justification:

All attributes depend fully on InventoryID.

Result:

The table satisfies 2NF.

Third Normal Form (3NF)

Rule of 3NF:

A table must:

Already be in 2NF

Have no transitive dependency

Supplier Table:

Issue:

No transitive dependency exists.

Justification:

All non-key attributes depend only on SupplierID.

Result:

The table satisfies 3NF.

Medicines Table

Issue:

Supplier information was not duplicated inside the table.

Justification:

Supplier details are stored separately in the Supplier table and linked using SupplierID.

Result:

The table satisfies 3NF.

Customers Table

Issue:

No transitive dependency exists.

Justification

All attributes depend directly on CustomerID.

Result:

The table satisfies 3NF.

Orders Table

Issue:

Customer information was separated from order information.

Justification:

Customer details are stored in the Customers table and linked using CustomerID.

Result:

The table satisfies 3NF.

OrderDetails Table

Issue:

Medicine and order data were properly separated.

Justification:

The table only stores relationship-specific attributes such as Quantity and Price.

Result:

The table satisfies 3NF.

Inventory Table

Issue:

No transitive dependency exists.

Justification:

Inventory-related attributes depend directly on InventoryID.

Result:

The table satisfies 3NF.

Duplicate Removal and Schema Improvements

Changes Performed

Removed duplication between Orders and Medicines using OrderDetails

Separated supplier information from Medicines

Separated customer information from Orders

Maintained inventory in a dedicated table

Result:

The database structure became more efficient, consistent, and easier to maintain.

Final Normalized Tables

- Supplier
- Medicines
- Customers
- Orders
- OrderDetails
- Inventory

Duplicate Removal and Redundancy Check

Medicines Table

Issue Checked:

Possible duplication of supplier information inside the Medicines table.

Action Taken:

Supplier details such as SupplierName and ContactNumber were removed from the Medicines table and stored separately in the Supplier table.

Reason:

This prevents repeated supplier data for every medicine record and improves consistency.

Orders Table

Issue Checked:

Possible duplication of customer information inside Orders.

Action Taken:

Customer details were separated into the Customers table and linked using CustomerID.

Reason:

This avoids repeating customer data in every order.

OrderDetails Table

Issue Checked

Direct many-to-many relationship between Orders and Medicines.

Action Taken:

The OrderDetails table was created to handle the relationship.

Reason:

This prevents duplicate medicine records inside orders and maintains normalization.

Inventory Table

Issue Checked:

Stock information stored with medicine details.

Action Taken:

Inventory was maintained in a separate table.

Reason:

This improves maintainability and allows better inventory management.

Final Result:

The schema was reviewed for duplicate and redundant data. All unnecessary repetition was removed by separating entities into dedicated tables and using foreign key relationships.

3. **DataFlow**
   Wholesale Medicine Database System — Dataflow Description
   Introduction

   The Wholesale Medicine Database System manages the flow of medicine-related business data from suppliers to customers through a structured relational database. The system stores, processes, and retrieves information related to medicines, suppliers, inventory, customer orders, and order details.

   Data Entry Sources

   Data enters the system through multiple entities involved in the wholesale medicine business.

   1. Supplier Data

Supplier information is entered into the Supplier table. This includes:

Supplier Name
Contact Number
Address

Each supplier is assigned a unique SupplierID.

2. Medicine Data

Medicine details are stored in the Medicines table. Data includes:

Medicine Name
Brand
Category
Price
Expiry Date
Batch Number

Each medicine is linked to a supplier using SupplierID.

3. Customer Data

Customer information such as pharmacy names, owner names, phone numbers, and addresses is stored in the Customers table.

Each customer is uniquely identified using CustomerID.

4. Order Data

When a pharmacy places an order, the order information is inserted into the Orders table. The system stores:

Order Date
Total Amount
CustomerID

5. Order Details Data

The medicines included in each order are stored in the OrderDetails table. This table records:

Ordered medicine
Quantity
Price

This table acts as a bridge between Orders and Medicines.

6. Inventory Data

Stock information is maintained in the Inventory table. The system tracks:

Current stock quantity
Minimum stock level

Inventory helps monitor medicine availability and low-stock conditions.

Data Movement Through the Database

The system processes data through relationships between tables.

Suppliers provide medicines
Medicines are stored in inventory
Customers place orders
Orders contain multiple medicines through OrderDetails
Inventory is updated based on medicine availability

Foreign keys are used to maintain relationships and ensure data consistency throughout the system.

Output and Usage of Data

The database can generate useful outputs including:

Medicine stock reports
Supplier records

Customer order history
Low inventory alerts
Sales and order summaries
Expiry tracking reports

The processed data can also be used in future enhancements such as:

Sales analysis
Demand prediction
AI-based inventory forecasting
Conclusion

The dataflow of the Wholesale Medicine Database System ensures efficient movement of information between suppliers, inventory, and customers. The structured relational design improves data integrity, reduces redundancy, and supports smooth business operations.

Milestone 3

1. **SYNTHETIC DATASET SECTION**
   Synthetic datasets were generated using Mockaroo to simulate realistic wholesale medicine business operations.

Milestone 4

1. **DDL Section**
   CREATE DATABASE WholesaleMedicineDB;

   USE WholesaleMedicineDB;

   -- Supplier Table

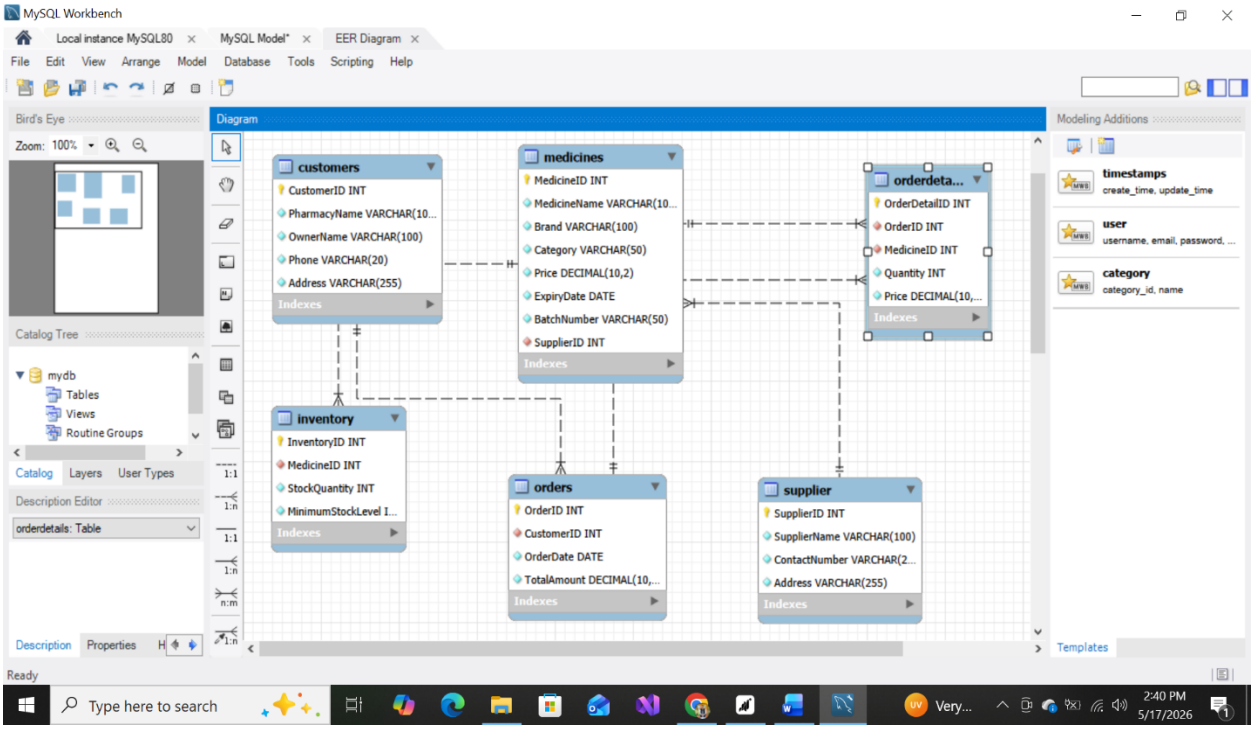
   CREATE TABLE Supplier (
   SupplierID INT PRIMARY KEY,

```
  SupplierName VARCHAR(100) NOT NULL,
  ContactNumber VARCHAR(20) NOT NULL UNIQUE,
  Address VARCHAR(255) NOT NULL
);


-- Medicines Table


CREATE TABLE Medicines (
  MedicineID INT PRIMARY KEY,
  MedicineName VARCHAR(100) NOT NULL,
  Brand VARCHAR(100) NOT NULL,
  Category VARCHAR(50) NOT NULL,
  Price DECIMAL(10,2) NOT NULL CHECK (Price > 0),
  ExpiryDate DATE NOT NULL,
  BatchNumber VARCHAR(50) NOT NULL UNIQUE,
  SupplierID INT NOT NULL,

  CONSTRAINT fk_supplier
    FOREIGN KEY (SupplierID)
    REFERENCES Supplier(SupplierID)
);

CREATE INDEX idx_supplierid
ON Medicines(SupplierID);


-- Customers Table


CREATE TABLE Customers (
  CustomerID INT PRIMARY KEY,
  PharmacyName VARCHAR(100) NOT NULL,
  OwnerName VARCHAR(100) NOT NULL,
  Phone VARCHAR(20) NOT NULL UNIQUE,
  Address VARCHAR(255) NOT NULL
);
```

```
-- Orders Table


CREATE TABLE Orders (
  OrderID INT PRIMARY KEY,
  CustomerID INT NOT NULL,
  OrderDate DATE NOT NULL,
  TotalAmount DECIMAL(10,2) NOT NULL CHECK (TotalAmount >= 0),

  CONSTRAINT fk_customer
    FOREIGN KEY (CustomerID)
    REFERENCES Customers(CustomerID)
);

CREATE INDEX idx_customerid
ON Orders(CustomerID);


-- OrderDetails Table


CREATE TABLE OrderDetails (
  OrderDetailID INT PRIMARY KEY,
  OrderID INT NOT NULL,
  MedicineID INT NOT NULL,
  Quantity INT NOT NULL CHECK (Quantity > 0),
  Price DECIMAL(10,2) NOT NULL CHECK (Price > 0),

  CONSTRAINT fk_order
    FOREIGN KEY (OrderID)
    REFERENCES Orders(OrderID),

  CONSTRAINT fk_medicine
    FOREIGN KEY (MedicineID)
    REFERENCES Medicines(MedicineID)
);
```

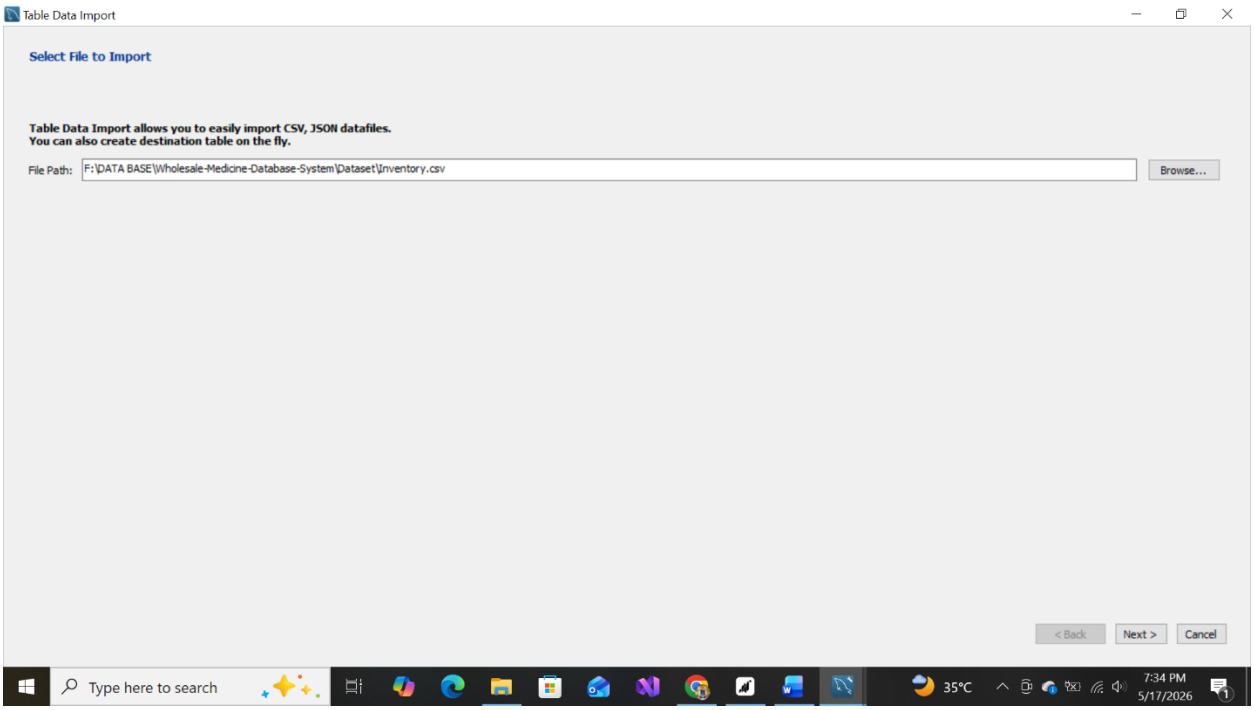
```
CREATE INDEX idx_orderid
ON OrderDetails(OrderID);

CREATE INDEX idx_medicineid
ON OrderDetails(MedicineID);


-- Inventory Table


CREATE TABLE Inventory (
  InventoryID INT PRIMARY KEY,
  MedicineID INT NOT NULL UNIQUE,
  StockQuantity INT NOT NULL CHECK (StockQuantity >= 0),
  MinimumStockLevel INT NOT NULL CHECK (MinimumStockLevel >= 0),

  CONSTRAINT fk_inventory_medicine
    FOREIGN KEY (MedicineID)
    REFERENCES Medicines(MedicineID)
);

CREATE INDEX idx_inventory_medicine
ON Inventory(MedicineID);
```

2. EER DIAGRAM VERIFICATION



Milestone 5

1. **Imported tables**



2. COUNT queries

```
110     ON Inventory(MedicineID);
111
112 •   SELECT COUNT(*) AS Supplier_Count FROM Supplier;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Supplier_Count
20

```
114 •   SELECT COUNT(*) AS Customers_Count FROM Customers;
115
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Customers_Count
61

```
115
116 •   SELECT COUNT(*) AS Medicines_Count FROM Medicines;
117
118 •   SELECT COUNT(*) AS Orders_Count FROM Orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Medicines_Count
100

3. **Join Query**

```
129 •    SELECT Orders.OrderID,
130          Customers.PharmacyName,
131          Orders.TotalAmount
132      FROM Orders
133      JOIN Customers
```

Result Grid | Filter Rows: | Ex

OrderID	PharmacyName	TotalAmount
1	Oyonder	36627.54
85	Oyonder	13839.16
52	Kazio	38115.08
88	Kazio	18824.28
91	Jatri	47509.11
31	Photolist	24868.26
56	Photolist	12166.49

Result 29

4. **NULL Query**

```
124 •    SELECT *
125      FROM Medicines
126      WHERE MedicineName IS NULL
127         OR Price IS NULL;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wra

MedicineID	MedicineName	Brand	Category	Price	ExpiryDate	BatchNumber	SupplierID
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

5. **Update Query**

3	105	19:40:30	UPDATE Medicines SET Price = Price + 100 WHERE Category = 'Tablet'	20 row(s) affected Rows matched: 20 Changed: 20 Warnings: 0	0.015 sec

6. **Update Query**

3	106	19:41:10	DELETE FROM Customers WHERE CustomerID = 59 AND CustomerID NOT IN (SELECT CustomerID FROM Orders)	0 row(s) affected	0.000 sec

Github Repository:

https://github.com/SyedSoban5/Wholesale-Medicine-Database-System

Conclusion:

The Wholesale Medicine Database System was successfully designed and implemented using normalization, relational schema design, SQL DDL/DML operations, and structured datasets. The project demonstrates effective database management for a real-world wholesale medicine business environment.